

G8 + P8 Preparation Checklist

PART I — COMMON FOUNDATION: G8 + P8

1. Programming, Data Structures and Algorithms

- C fundamentals: variables, data types, operators, control flow, functions, recursion, arrays, strings, pointers, structures and dynamic memory allocation.
- Data structures: arrays, linked lists, stacks, queues, trees, binary search trees, heaps, hashing and graphs.
- Algorithms: searching, sorting, divide and conquer, greedy methods, dynamic programming, graph traversal, shortest paths and minimum spanning trees.
- Complexity analysis: Big-O, Big-Theta, Big-Omega, best, average and worst cases; time and space complexity.
- Recurrences: substitution method, recursion tree and Master theorem.
- Computability and complexity basics: P, NP, NP-hard, NP-complete, polynomial-time reductions, SAT, 3-SAT, clique, vertex cover and Hamiltonian cycle.
- Recursion and backtracking: subsets, permutations, N-Queens, Sudoku and pruning.
- Bit manipulation: AND, OR, XOR, bit masks, counting set bits and power-of-two checks.

2. DBMS and SQL

- Database fundamentals: ER model, entities, attributes, relationships, cardinality, participation, weak entities, relational model and keys.
- SQL: SELECT, WHERE, DISTINCT, ORDER BY, GROUP BY, HAVING, joins, subqueries, correlated subqueries, EXISTS, NOT EXISTS, set operations, CASE, CTEs and window functions.
- Relational theory: relational algebra, tuple relational calculus, domain relational calculus, functional dependencies, attribute closure and candidate keys.
- Normalization: 1NF, 2NF, 3NF, BCNF, lossless decomposition and dependency preservation.
- Transactions: ACID properties, schedules, conflict and view serializability, recoverability, locking, two-phase locking, deadlocks and recovery.
- Concurrency and recovery: timestamp ordering, strict and rigorous 2PL, multiple-granularity locking, isolation levels, write-ahead logging and checkpoints.
- Indexing and file organization: primary and secondary indexes, B-trees, B+ trees, hashing, heap and sorted files, clustered and non-clustered indexes, B+ tree order and height numericals.
- SQL practice: top-N per group, running totals, self joins, gaps and islands, query debugging and explaining query output.

3. Operating Systems

- Processes, process control blocks, process states, context switching, inter-process communication, threads and multithreading.
- CPU scheduling: FCFS, SJF, SRTF, priority scheduling and Round Robin; waiting, turnaround and response-time numericals.
- Synchronization: critical sections, race conditions, mutexes, semaphores, monitors, producer-consumer, readers-writers and dining philosophers.
- Deadlocks: necessary conditions, resource-allocation graphs, prevention, avoidance, Banker's algorithm, detection and recovery.
- Memory management: paging, segmentation, page tables, TLB, virtual memory, page faults, FIFO, LRU and Optimal page replacement.

- Advanced memory topics: thrashing, working-set model, Belady's anomaly, multilevel paging and effective access time numericals.
- File systems, file allocation, inodes, disk scheduling, system calls, interrupts, traps, user mode and kernel mode, I/O subsystem and DMA basics.

4. Computer Networks

- OSI and TCP/IP models; packet switching and circuit switching.
- Data link layer: framing, error detection, CRC, Ethernet, MAC, CSMA/CD, hubs, bridges, switches, VLANs and spanning tree.
- Reliable data transfer: Go-Back-N, Selective Repeat and sliding-window numericals.
- Network layer: IPv4 and IPv6, addressing, subnetting, CIDR, routing, ARP, ICMP, fragmentation and NAT.
- Routing: distance vector, link state, RIP, OSPF and BGP.
- Transport layer: TCP, UDP, ports, reliability, flow control, congestion control and three-way handshake.
- Application layer: HTTP/HTTPS, DNS, DHCP, FTP, SMTP, POP and IMAP.
- Network security basics: symmetric and asymmetric cryptography, DES/AES, RSA, Diffie–Hellman, hashing, digital signatures, certificates, PKI, TLS, firewalls, IPsec, DoS, spoofing and man-in-the-middle attacks.
- Numericals: transmission and propagation delay, throughput, window size and subnetting.

5. OOP and Core Technical Interviews

- Classes, objects, constructors, access modifiers, encapsulation, abstraction, inheritance and polymorphism.
- Overloading versus overriding, abstract classes, interfaces, virtual functions and exception handling.
- Composition versus inheritance, SOLID principles and common why/how interview questions.
- Low-level design: Singleton, Factory, Observer, Strategy, Builder and Decorator patterns; UML class diagrams; parking lot, elevator, LRU cache, booking system and rate limiter designs.

PART II — G8: G8 PREPARATION

6. Engineering Mathematics

- Discrete mathematics: logic, sets, relations, functions, counting, pigeonhole principle, inclusion-exclusion, recurrence relations, graph theory, trees, graph colouring, Euler and Hamiltonian graphs, and algebraic structures.
- Additional discrete topics: partial orders, Hasse diagrams, lattices, groups and subgroups, generating functions, linear recurrences, planar graphs and matching.
- Linear algebra: matrices, determinants, rank, inverse, systems of equations, Gaussian elimination, eigenvalues, eigenvectors and vector spaces.
- Calculus: limits, continuity, differentiation, partial derivatives, maxima and minima, integration, definite and multiple integrals, Rolle's theorem, Mean Value Theorem, Taylor and Maclaurin series, and L'Hôpital's rule.
- Probability and statistics: conditional probability, Bayes' theorem, independence, random variables, PMF/PDF/CDF, expectation, variance, covariance, correlation and common distributions.
- Distributions and testing: Bernoulli, binomial, Poisson, geometric, uniform, exponential and normal distributions; their means and variances; basic hypothesis testing.

7. Digital Logic

- Number systems and conversions, signed and unsigned representation, Boolean algebra, Boolean laws and De Morgan's laws.
- SOP/POS, canonical forms, Karnaugh maps, AND, OR, NOT, NAND, NOR, XOR and XNOR gates.
- Combinational circuits: adders, subtractors, multiplexers, demultiplexers, encoders, decoders, comparators, ROM, PLA and PAL.
- Hazards and implementation: static and dynamic hazards; implementing functions using multiplexers and decoders.
- Sequential circuits: latches, SR/JK/D/T flip-flops, registers, counters, Mealy and Moore machines, state diagrams and tables, state minimization, synchronous and asynchronous counter design, and excitation tables.

8. Computer Organization and Architecture

- CPU, ALU, control unit, registers, buses, instruction cycle, data representation, signed numbers, 1's and 2's complement, floating point and computer arithmetic.
- IEEE 754: single and double precision, bias, normalized and denormalized numbers, special values and rounding.
- Instruction formats, addressing modes, RISC versus CISC, datapath and control.
- Hardwired versus microprogrammed control and microinstruction formats.
- Pipelining: speedup, structural, data and control hazards; CPI with stalls, branch penalty, forwarding, speedup and throughput numericals.
- Memory hierarchy, cache mapping, cache hit/miss, replacement and write policies, interrupts and DMA.
- I/O: programmed I/O, interrupt-driven I/O, DMA modes, I/O interfaces and average memory access time numericals.

9. Theory of Computation

- DFA, NFA, epsilon-NFA, conversions and minimization.
- Regular expressions and languages, closure properties and pumping lemma.
- Context-free grammars, derivations, parse trees, ambiguity, CNF and GNF.
- Pushdown automata, CFG-to-PDA conversion, Turing machines, decidability, undecidability, halting problem and reductions.
- Language hierarchy: Chomsky hierarchy; regular, DCFL, CFL, CSL, recursive and recursively enumerable classes; closure and decidability tables.
- Advanced topics: pumping lemma for CFLs, DPDA versus NPDA, Rice's theorem and Post correspondence problem.

10. Compiler Design

- Lexical analysis, tokens, lexemes, regular expressions and finite automata.
- CFGs, parse trees, top-down and bottom-up parsing, LL, LR, SLR, LALR and CLR parsing.
- Semantic analysis, type checking and symbol tables.
- Syntax-directed translation: syntax-directed definitions, S-attributed and L-attributed definitions, and annotated parse trees.
- Intermediate code: three-address code, quadruples and triples.
- Runtime environments: activation records, stack versus heap allocation, parameter passing, and static versus dynamic scope.
- Optimization: constant folding, dead-code elimination, common-subexpression elimination and loop optimization.
- Data-flow analysis: basic blocks, control-flow graphs, liveness, reaching definitions, available expressions and constant propagation.
- Code generation, instruction selection and register allocation.

11. General Aptitude for G8

- Quantitative aptitude: number systems, percentages, ratios, averages, profit and loss, interest, time and work, speed and distance, permutations, combinations, probability and statistics.
- Reasoning: statements, conclusions, assumptions, syllogisms, series, arrangements, puzzles and data interpretation.
- Verbal ability: grammar, vocabulary, sentence completion, reading comprehension and inference.

12. G8 Practice and Revision

- Solve topic-wise previous-year questions, then subject-wise sets, sectional tests and full-length mock tests.
- Maintain an error log for conceptual gaps, calculation errors and avoidable mistakes.
- Use spaced revision, formula sheets, concise concept notes and timed numerical practice.
-

PART III — P8: P8 PREPARATION

13. DSA for Coding Rounds

- Arrays: two pointers, sliding window, prefix sums, Kadane's algorithm and matrices.
- Strings: frequency maps, anagrams, palindromes, pattern matching, KMP, Z-algorithm, Rabin–Karp rolling hash and tries.
- Linked lists: reversal, fast/slow pointers, cycle detection, merging and intersection.
- Stacks and queues: monotonic stack, next greater element, expression problems, deque, sliding-window maximum and priority queue.
- Binary search: first/last occurrence, rotated arrays and search on answer.
- Trees and heaps: traversals, height, diameter, lowest common ancestor, BST operations, top-K and median problems.
- Advanced structures when relevant: segment trees, Fenwick trees and balanced BST concepts.
- Graphs: BFS, DFS, cycle detection, topological sorting, shortest paths, minimum spanning trees and disjoint-set union.
- Greedy and dynamic programming: intervals, scheduling, knapsack, subsequences, grids and state transitions.
- Advanced practice: backtracking, bitmask DP, TSP-style bitmask DP and tree DP.
- Practice: company-style online assessments, timed rounds, LeetCode contests and Codeforces Div. 3/4 contests.

14. Programming Language and Coding Skills

- Choose one main language: Python, C++ or Java. Be able to implement common data structures and algorithms without relying on memorized templates.
- Python: syntax, functions, lists, tuples, sets, dictionaries, classes, exceptions, iterators, generators, lambda, collections, heapq and bisect.
- C++ if selected: STL vector, map, unordered_map, set, priority_queue, algorithms, iterators, lambdas and smart pointers.
- Java if selected: Collections framework, ArrayList, HashMap, TreeMap, PriorityQueue, streams and generics.
- Write readable code, debug independently, test edge cases, explain the approach and state time and space complexity.
- Concurrency basics: threads, locks, thread pools, producer-consumer implementation, deadlock avoidance, asynchronous programming versus multithreading.

15. Software Development

- HTML: semantic elements, forms, tables, accessibility and DOM basics.
- CSS: selectors, box model, Flexbox, Grid, responsive design, positioning and media queries.
- JavaScript: variables, functions, arrays, objects, ES6+, destructuring, promises, async/await, fetch, DOM, events and modules.
- React: components, props, state, hooks, routing, forms, API integration, authentication and state-management basics.
- Backend: Node.js, npm, Express, routing, middleware, controllers, REST APIs, validation and error handling.
- Web/API fundamentals: HTTP methods and status codes, JSON, headers, JWT, cookies, authentication versus authorization.
- Software engineering basics to add: SDLC, Agile/Scrum, requirements, design, code review, unit/integration/system testing, debugging and basic test-case design.

16. Databases for Development

- PostgreSQL or MySQL: schema design, relationships, constraints, joins, indexes, transactions and query optimization.
- MongoDB: collections, documents, CRUD, queries, aggregation, indexes and Mongoose.
- Know when relational databases and document databases are appropriate.
- Caching and operations: Redis basics, EXPLAIN plans, connection pooling and database migrations.

17. Git, Linux, DevOps and Deployment

- Git: init, clone, add, commit, push, pull, branches, merge, rebase basics, stash and conflict resolution.
- GitHub: repositories, pull requests, README files, issues and collaboration workflows.
- Linux: filesystem, common commands, permissions, processes, environment variables, SSH and Bash basics.
- Docker: images, containers, Dockerfiles, Compose, volumes, networks and environment variables.

- Deployment: servers, DNS, HTTPS, logs, cloud basics and CI/CD fundamentals.
- Practical additions: environment configuration, secrets handling, basic automated tests and troubleshooting deployment failures.

18. System Design

- High-level design: client-server architecture, vertical and horizontal scaling, load balancing, CDN, caching, cache invalidation, message queues, API gateways and rate limiting.
- Data layer: SQL versus NoSQL, replication, sharding, partitioning, indexing, consistency models, CAP theorem and ACID versus BASE.
- Reliability: availability, fault tolerance, monitoring, logging and back-of-the-envelope estimation.
- Practice designs: URL shortener, chat application, notification service, news feed, file storage and rate limiter.
- Connect high-level design with low-level design, design patterns and data-structure choices.

19. Projects, Resume and Profile

- Build two or three substantial projects rather than many small CRUD-only projects.
- Full-stack project: frontend, backend, database, authentication, testing and deployment.
- ML/AI project: data collection, exploratory analysis, feature engineering, model, evaluation, API, frontend and deployment.
- GenAI project: real problem, LLM/RAG/agent approach, evaluation, failure analysis and safety considerations.
- Be ready to explain architecture, technology choices, algorithms, database design, APIs, challenges, trade-offs, measured results and deployment.
- Maintain a concise one-page resume and a clear GitHub profile with readable READMEs and reproducible instructions.

20. Aptitude, Interviews and Online Assessments

- Quantitative aptitude: percentages, ratios, averages, profit and loss, time and work, speed and distance, probability, permutations and combinations, and number systems.
- Reasoning: series, arrangements, puzzles, blood relations, directions and syllogisms.
- Verbal ability: reading comprehension, grammar and vocabulary.
- Technical interviews: DSA, DBMS, OS, computer networks, OOP, SQL and detailed discussion of personal projects.
- Behavioural interviews: introduction, teamwork, conflict, failure, leadership, learning from mistakes and role/company questions.
- Practice timed coding rounds, aptitude mocks, technical mock interviews and concise project explanations.
- Before each drive, review the employer's official role description and assessment format; requirements differ by role and company.

21. ML and AI Profile (for AI/ML-oriented roles)

- Python and tools: NumPy, Pandas, Matplotlib, scikit-learn and PyTorch.
- Machine learning: regression, classification, KNN, Naive Bayes, decision trees, random forests, SVM, clustering, PCA and ensemble methods.
- Evaluation: train/test split, cross-validation, accuracy, precision, recall, F1, ROC-AUC, confusion matrix, bias/variance and overfitting.
- Deep learning: neural networks, activation functions, backpropagation, gradient descent, CNNs, RNNs/LSTMs and Transformers.
- Generative AI: LLMs, tokenization, embeddings, attention, RAG, vector databases, fine-tuning, LoRA, agents and evaluation.
- Applied ML: feature engineering, class imbalance, hyperparameter tuning and experiment tracking.
- MLOps: model serving with FastAPI, Docker for models, monitoring, drift detection and model/version management.
- LLM evaluation and safety: evaluation sets, LLM-as-judge limitations, hallucination checks, guardrails and prompt-injection basics.